

Ingeniería de Software/Informática - Fases del desarrollo de soluciones informáticas.

Para transformar una idea en un producto de software, ya sea un juego o cualquier otra aplicación, se sigue un ciclo de vida que, en términos generales, abarca desde la concepción hasta la entrega.

1. Especificación (Requisitos)

Esta es la fase de **qué construir**. Aquí se define el alcance del proyecto y las funcionalidades.

- **Identificación de las necesidades:** Se determina el problema que el software resolverá o la oportunidad que aprovechará. En tu caso del juego, sería la idea central, el género, la experiencia de juego que quieres ofrecer, etc.
 - **Recopilación de requisitos:** Se juntan todas las peticiones y expectativas de los usuarios y stakeholders. Esto incluye:
 - **Requisitos funcionales:** ¿Qué debe hacer el software? (Ej: “El jugador debe poder elegir entre 2 enemigos”, “Las cartas deben tener efectos de daño”).
 - **Requisitos no funcionales:** ¿Cómo debe funcionar? (Ej: rendimiento, seguridad, usabilidad, escalabilidad. Para un juego, podría ser “Debe correr a 60 FPS en X dispositivo”).
 - **Análisis de requisitos:** Se revisan, organizan y validan los requisitos para asegurar que sean claros, completos, consistentes y factibles. Se resuelven ambigüedades y conflictos.
 - **Documentación de requisitos:** Se formalizan los requisitos en documentos como el Documento de Especificación de Requisitos (DER) o historias de usuario (en metodologías ágiles). Para tu juego, esto podría incluir un Game Design Document (GDD) detallado.
-

2. Diseño

La fase de diseño responde al **cómo construir**. Aquí se planifica la arquitectura y la estructura del software basándose en los requisitos.

- **Diseño arquitectónico:** Se define la estructura general del sistema, cómo se dividirán sus componentes y cómo interactuarán entre sí. Aquí se aplican tus conocimientos de arquitecturas de software. Para un

juego, esto podría ser la división entre la lógica del juego, la interfaz de usuario, la gestión de assets, etc.

- **Diseño de componentes (o módulos):** Cada componente de la arquitectura se diseña en detalle. Se definen sus responsabilidades, interfaces y algoritmos. Aquí es donde los **patrones de diseño** que conoces son cruciales para crear soluciones elegantes y reutilizables (ej: patrón Observer para la notificación de eventos de cartas, Factory para la creación de diferentes tipos de cartas).
 - **Diseño de datos:** Si aplica, se define la estructura de las bases de datos o el almacenamiento de datos (ej: cómo se guardará el progreso del jugador, las cartas desbloqueadas).
 - **Diseño de interfaz de usuario (UI/UX):** Se planifica cómo el usuario interactuará con el software. Esto incluye wireframes, mockups y prototipos para visualizar la experiencia de usuario. En tu juego, sería el diseño de la interfaz de combate, los menús, la visualización de cartas, etc.
 - **Evaluación del diseño:** Se revisa el diseño para asegurar que cumple con los requisitos y que es robusto, eficiente y mantenible.
-

3. Desarrollo (Implementación)

Esta es la fase de **construir el software** siguiendo los diseños y especificaciones.

- **Codificación:** Se escribe el código fuente del software utilizando el lenguaje de programación y las herramientas seleccionadas (en tu caso, C# en Unity). Se aplican las **buenas prácticas de programación** que conoces para asegurar un código limpio, legible y eficiente.
 - **Pruebas unitarias:** A medida que se desarrollan los componentes, se prueban individualmente para asegurar que cada parte funcione correctamente de forma aislada.
 - **Integración:** Los diferentes módulos o componentes desarrollados se combinan y prueban juntos para asegurar que funcionen como un sistema unificado.
 - **Gestión de versiones:** Se utiliza un sistema de control de versiones (como Git) para gestionar los cambios en el código, permitir la colaboración entre equipos y tener un historial de versiones.
 - **Depuración:** Se identifican y corrigen los errores (bugs) que surgen durante la codificación y las pruebas.
-

4. Pruebas (Verificación y Validación)

Aunque las pruebas unitarias se hacen en desarrollo, esta fase se enfoca en probar el sistema completo.

- **Pruebas de integración:** Se verifica que los módulos interactúen correctamente.
 - **Pruebas de sistema:** Se prueba el software como un todo para asegurar que cumple con todos los requisitos funcionales y no funcionales.
 - **Pruebas de aceptación del usuario (UAT):** Los usuarios finales o clientes prueban el software para validar que satisface sus necesidades y expectativas. En tu juego, esto sería la fase de *playtesting* intensivo.
 - **Pruebas de rendimiento:** Se evalúa la velocidad, escalabilidad y estabilidad del sistema bajo diferentes cargas (ej: cómo se comporta tu juego con muchas animaciones o efectos simultáneos).
-

5. Despliegue y Mantenimiento

Una vez que el software está desarrollado y probado, se pone a disposición de los usuarios y se gestiona su vida útil.

- **Despliegue:** El software se instala y configura en el entorno de producción (servidores, tiendas de aplicaciones, etc.) para que los usuarios puedan acceder a él. Para tu juego, sería publicarlo en plataformas como Steam, Google Play, o la App Store.
 - **Mantenimiento:** Incluye la corrección de errores que surjan después del despliegue, la implementación de mejoras o nuevas funcionalidades (evolución), y la adaptación a nuevos entornos o tecnologías. Este es un ciclo continuo durante toda la vida del software.
-

Estos pasos son iterativos y flexibles. En metodologías ágiles, por ejemplo, los ciclos son más cortos y los pasos se repiten con frecuencia, entregando versiones incrementales del software.